Author: Suhas Suresha & Dewang Sultania

SECTION 1

The model service contract

- **Central choke point:** Every generation, classification, or structured extraction flows through gRPC operations that define the platform's inference promise.
- **Core chat:** `Chat` and `ChatStream` accept model choice, message roles, tools, and generation config while returning content, usage, and finish metadata.
- **Discovery:** Listing models and capabilities lets workflows avoid stale hardcoded names and pick windows or modalities at runtime.
- **Prompt registry:** Named, versioned system prompts decouple legal-reviewed personas from workflow code and enable eval comparisons.
- **Custom model registration:** Internal fine-tunes and OpenAI-compatible endpoints become first-class models with health checks and unified billing fields.

SECTION 2

Provider abstraction and adapters

- **Heterogeneous APIs:** Vendors disagree on system prompts, response envelopes, token field names, and streaming—direct integration locks teams to one vendor.
- **Adapter pattern:** A `ModelProvider` interface plus per-vendor classes translate messages, map parameters, normalize responses, map errors, and handle structured-output modes.
- **OpenAI-shaped canonical messages:** The platform standardizes role/content arrays (plus tool fields) so Session storage and most adapters share one shape.
- **Self-hosted parity:** vLLM-style servers reuse OpenAI-compatible paths, enabling on-prem fallbacks without forked application logic.

SECTION 3

Streaming

- **Perceived latency:** Progressive token delivery improves UX via time-to-first-token even when total generation time is unchanged.
- **Pipeline layers:** Provider SSE → adapter chunks → gRPC `ChatStream` → gateway SSE gives one `ChatChunk` contract to clients.
- **Mid-stream failures:** Partial text may remain valid while a terminal chunk signals error finish reasons; fallbacks typically apply before streaming starts to avoid jarring voice changes.

SECTION 4

Resilience: retries and fallbacks

- **RetryConfig:** Applications specify which transient failures (`RATE_LIMIT`, `TIMEOUT`, `UNAVAILABLE`) deserve bounded exponential backoff; permanent errors fail fast.
- **FallbackConfig:** Ordered alternate providers each get their own retry loop; responses record which provider actually answered for cost and quality attribution.
- **Policy tuning:** Patient-facing assistants favor short retries and deep chains; batch pipelines may disable fallbacks to surface poison inputs instead of masking them.

SECTION 5

Routing strategies

- **Pre-call selection:** When `model` is empty, `RoutingConfig` applies cost, load, feature, or combined strategies maintained with platform-wide spend and capability state.
- **Cost-aware:** Budget proximity steers traffic toward cheaper or local models while reserving frontier models for complex tasks.
- **Load-based:** In-flight counters spread requests across providers to delay rate-limit cascades into fallback spend.
- **Feature-based:** Required capabilities (vision, tools, large context) intersect provider matrices; impossible requests fail clearly instead of hitting incapable endpoints.

SECTION 6

Rate limiting and caching

- **Platform rate limits:** Org-level caps on requests, tokens, concurrency, and scope protect budgets before vendor calls; platform limits should not silently trigger cross-provider fallbacks that defeat spend controls.
- **Response cache:** Hash-identical messages plus config can return zero-cost, sub-millisecond answers for deterministic FAQs when temperature allows repetition.
- **Prompt/prefix cache:** Vendor-side reuse of stable prefixes cuts token charges on remaining API calls; adapters pass provider-specific cache hints while keeping application caches separate.

SECTION 7

Observability and SDK integration

- **Per-request metrics:** Token usage, actual provider/model after failover, latency (including TTFT), error taxonomy, cache path, and workflow or user attribution feed the Observability Service.
- **Actionable finance:** Aggregated dimensions answer which feature drove a spike instead of mystery invoices like Sam's early experience.
- **`ModelClient`:** Lazy SDK client converts Python calls to protobuf, invokes `stub.Chat` or `ChatStream` through the gateway, and returns typed responses with usage intact.
- **Boundary discipline:** Sarah configures models and policies at the workflow; the service executes translation, routing, resilience, caching, and metering so keys and vendor SDKs stay off application images.